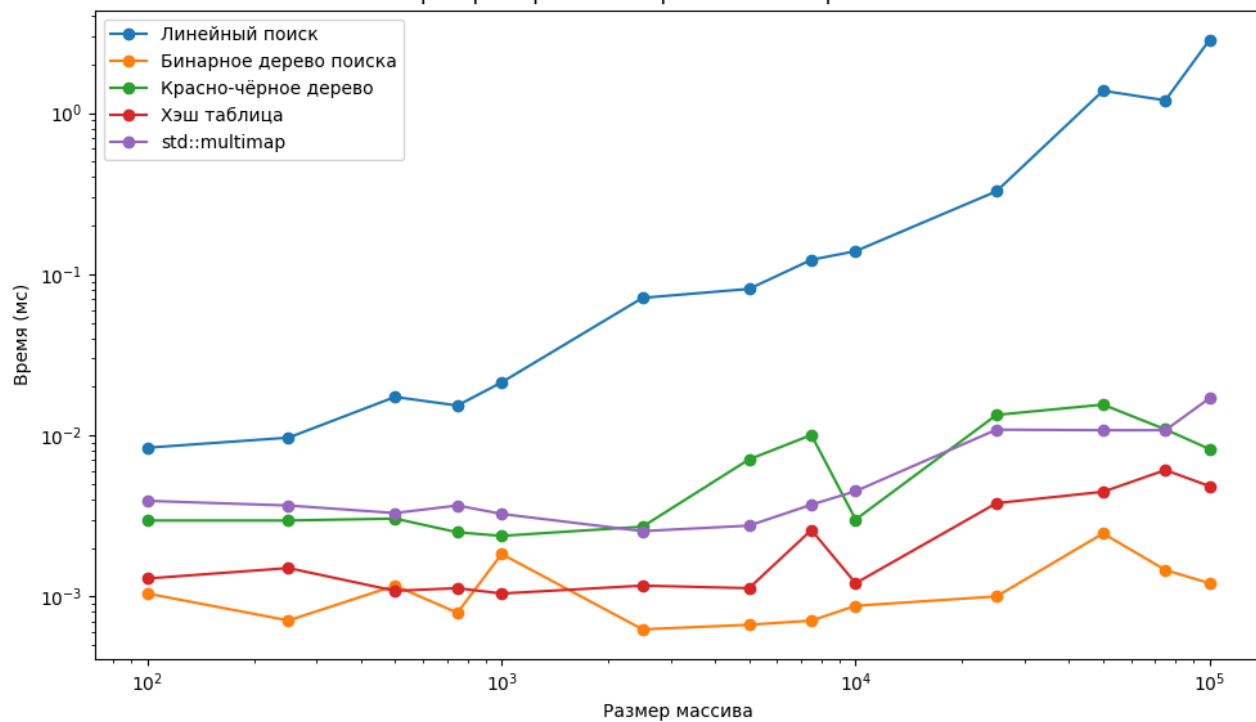
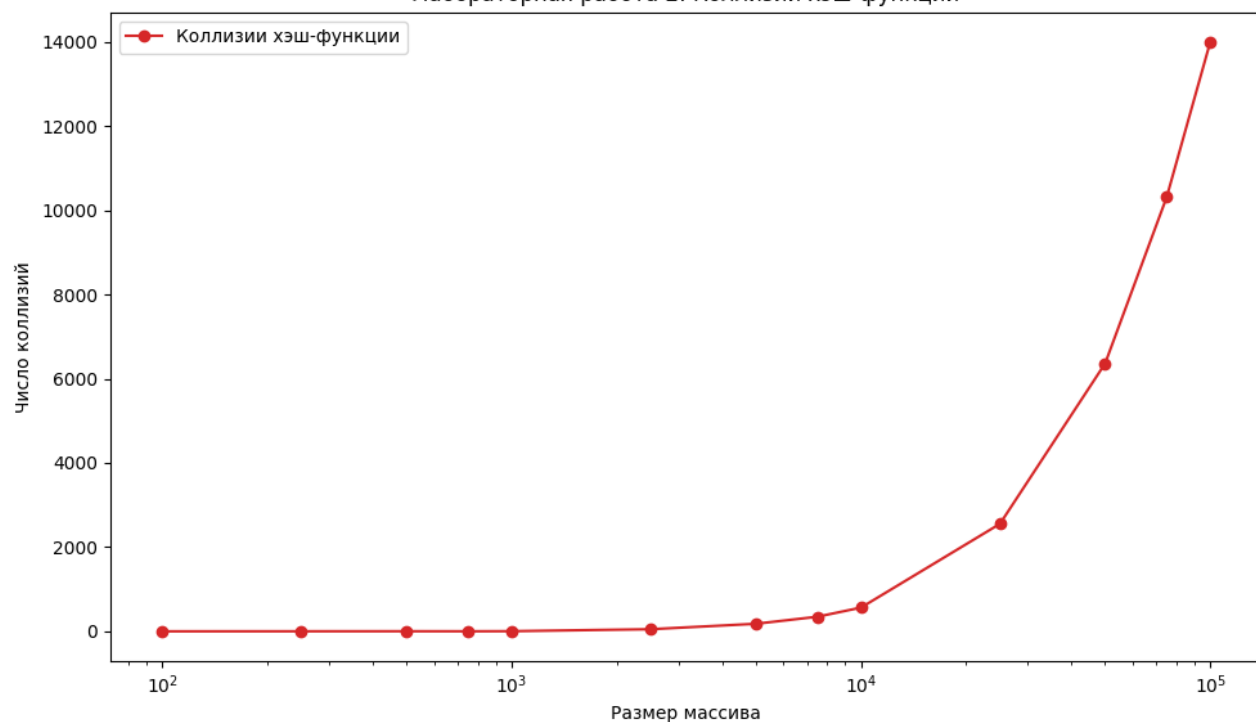


Лабораторная работа 2

Лабораторная работа 2: Сравнение алгоритмов поиска



Лабораторная работа 2: Коллизии хэш-функции



Репозиторий: <https://github.com/diduk001/programming-techniques/tree/main/practice2>

Лабораторная работа 2: «Алгоритмы поиска данных»

Общее задание

1) Реализовать поиск заданного элемента в массиве (необходимо найти ВСЕ вхождения) объектов по ключу в соответствии с вариантом (структурой данных из ЛР1, ключом является первое НЕ числовое поле объекта) следующими методами:

- линейный поиск
- с помощью бинарного дерева поиска
- с помощью красно-черного дерева
- с помощью хэш таблицы

Все методы необходимо реализовать самостоятельно.

Предполагается, что в исходном массиве данных ключи не уникальны.

2) Для хэш таблицы необходимо реализовать хэш функцию и метод разрешения коллизий. Подсчитать число коллизий хэш функции и построить график зависимости от размерности массива.

3) Выполнить поиск не менее 10 раз на массивах разных размерностей от 100 до 1000000. Засечь (программно) время поиска для всех способов. По полученным точкам построить сравнительные графики зависимости времени поиска от размерности массива.

4) Записать входные данные в ассоциативный массив multimap и сравнить время поиска по ключу в нем с временем поиска из п.3. Добавить данные по времени поиска в ассоциативном массиве в общее сравнение с остальными способами и построить график зависимости времени поиска от размерности массива.

5) Требования к отчету аналогичны работе 1.

Вариант 7

Данные: Массив данных о сборной олимпийской команде: ФИО, возраст, рост, вес, вид спорта (сравнение по полям – вид спорта, ФИО, возраст)

Лабораторная работа №2

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 BinarySearchTree Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Function Documentation	6
3.1.2.1 insert()	6
3.1.2.2 insertRec()	6
3.1.2.3 search()	7
3.1.2.4 searchRec()	7
3.1.3 Member Data Documentation	7
3.1.3.1 root	7
3.2 BSTNode Struct Reference	8
3.2.1 Detailed Description	8
3.2.2 Constructor & Destructor Documentation	8
3.2.2.1 BSTNode()	8
3.2.3 Member Data Documentation	8
3.2.3.1 left	8
3.2.3.2 originalIndex	9
3.2.3.3 right	9
3.2.3.4 value	9
3.3 FullName Struct Reference	9
3.3.1 Detailed Description	10
3.3.2 Member Function Documentation	10
3.3.2.1 operator!=(())	10
3.3.2.2 operator<()	10
3.3.2.3 operator<=()	11
3.3.2.4 operator==(())	11
3.3.2.5 operator>()	11
3.3.2.6 operator>=()	12
3.4 HashTable Struct Reference	12
3.4.1 Detailed Description	13
3.4.2 Member Function Documentation	13
3.4.2.1 insert()	13
3.4.3 Member Data Documentation	13
3.4.3.1 buckets_indices	13
3.4.3.2 collisions_cnt	13
3.4.3.3 total_insertions	14

3.5 RNode Struct Reference	14
3.5.1 Detailed Description	14
3.5.2 Constructor & Destructor Documentation	15
3.5.2.1 RNode()	15
3.5.3 Member Data Documentation	15
3.5.3.1 color	15
3.5.3.2 left	15
3.5.3.3 originalIndex	15
3.5.3.4 parent	15
3.5.3.5 right	16
3.5.3.6 value	16
3.6 RedBlackTree Class Reference	16
3.6.1 Detailed Description	16
3.6.2 Member Function Documentation	17
3.6.2.1 fix_insert()	17
3.6.2.2 insert()	17
3.6.2.3 rotate_left()	17
3.6.2.4 rotate_right()	18
3.6.2.5 search_all()	18
3.6.3 Member Data Documentation	18
3.6.3.1 root	18
3.7 Sportsman Struct Reference	18
3.7.1 Detailed Description	19
3.7.2 Constructor & Destructor Documentation	19
3.7.2.1 Sportsman()	19
3.7.3 Member Function Documentation	20
3.7.3.1 operator!=()	20
3.7.3.2 operator<()	20
3.7.3.3 operator<=()	21
3.7.3.4 operator==(())	21
3.7.3.5 operator>()	21
3.7.3.6 operator>=()	22
3.7.3.7 to_csv()	22
3.7.4 Member Data Documentation	22
3.7.4.1 age	22
3.7.4.2 full_name	23
3.7.4.3 height_cm	23
3.7.4.4 sport	23
3.7.4.5 weight_kg	23
4 File Documentation	25

4.1 src/search.cpp File Reference	25
4.1.1 Detailed Description	26
4.1.2 Enumeration Type Documentation	26
4.1.2.1 Color	26
4.1.3 Function Documentation	26
4.1.3.1 hash_sportsman()	26
4.2 src/search.h File Reference	27
4.2.1 Detailed Description	27
4.3 search.h	27
4.4 src/sportsman.h File Reference	27
4.4.1 Detailed Description	28
4.5 sportsman.h	28
4.6 src/utils.h File Reference	30
4.6.1 Detailed Description	30
4.6.2 Function Documentation	30
4.6.2.1 from_csv_file()	30
4.6.2.2 measure_time_ms()	31
4.6.2.3 to_csv_file()	31
4.7 utils.h	32
Index	33

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BinarySearchTree	Двоичное дерево поиска	5
BSTNode	Узел двоичного дерева поиска	8
FullName	ФИО спортсмена	9
HashTable	Хеш-таблица для хранения спортсменов и их индексов	12
RBNode	Узел красно-черного дерева	14
RedBlackTree	Красно-черное дерево	16
Sportsman	Структура данных, описывающая спортсмена	18

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/search.cpp	Реализация методов поиска спортсменов в массиве	25
src/search.h	Объявление методов поиска спортсменов в массиве	27
src/sportsman.h	Структура данных, описывающая спортсмена	27
src/utils.h	Вспомогательные функции	30

Chapter 3

Class Documentation

3.1 BinarySearchTree Struct Reference

Двоичное дерево поиска

Public Member Functions

- `BinarySearchTree ()`
Конструктор, который инициализирует пустое дерево
- `void insert (const Sportsman &value, int index)`
Вставляет спортсмена в дерево с сохранением индекса из исходного массива
- `BSTNode * insertRec (BSTNode *node, const Sportsman &value, int index)`
Рекурсивная функция для вставки спортсмена в дерево
- `void search (const Sportsman &target, std::vector< int > &indices)`
Ищет спортсмена в дереве
- `void searchRec (BSTNode *node, const Sportsman &target, std::vector< int > &indices)`
Рекурсивная функция для поиска спортсмена в дереве

Public Attributes

- `BSTNode * root`

3.1.1 Detailed Description

Двоичное дерево поиска

3.1.2 Member Function Documentation

3.1.2.1 insert()

```
void BinarySearchTree::insert (  
    const Sportsman & value,  
    int index) [inline]
```

Вставляет спортсмена в дерево с сохранением индекса из исходного массива

Parameters

 [value] 	Спортсмен для вставки
 [index] 	Индекс спортсмена в исходном массиве

3.1.2.2 insertRec()

```
BSTNode * BinarySearchTree::insertRec (  
    BSTNode * node,  
    const Sportsman & value,  
    int index) [inline]
```

Рекурсивная функция для вставки спортсмена в дерево

Parameters

 [node] 	Узел дерева для вставки
 [value] 	Спортсмен для вставки
 [index] 	Индекс спортсмена в исходном массиве

Returns

Указатель на узел после вставки

3.1.2.3 search()

```
void BinarySearchTree::search (
    const Sportsman & target,
    std::vector< int > & indices) [inline]
```

Ищет спортсмена в дереве

Parameters

$\leftrightarrow$ [target] $\leftrightarrow$	Спортсмен для поиска
$\leftrightarrow$ [indices] $\leftrightarrow$	Вектор для хранения индексов найденных спортсменов

3.1.2.4 searchRec()

```
void BinarySearchTree::searchRec (
    BSTNode * node,
    const Sportsman & target,
    std::vector< int > & indices) [inline]
```

Рекурсивная функция для поиска спортсмена в дереве

Parameters

[node]	Узел дерева для поиска
$\leftrightarrow$ [target] $\leftrightarrow$	Спортсмен для поиска
$\leftrightarrow$ [indices] $\leftrightarrow$	Вектор для хранения индексов найденных спортсменов

3.1.3 Member Data Documentation

3.1.3.1 root

```
BSTNode* BinarySearchTree::root
```

Корень дерева

The documentation for this struct was generated from the following file:

- [src/search.cpp](#)

3.2 BSTNode Struct Reference

Узел двоичного дерева поиска

Public Member Functions

- [BSTNode](#) (const [Sportsman](#) &val, int idx)
Конструктор, который инициализирует узел на основе спортсмена и его индекса

Public Attributes

- [Sportsman](#) value
- int [originalIndex](#)
- [BSTNode](#) * [left](#)
- [BSTNode](#) * [right](#)

3.2.1 Detailed Description

Узел двоичного дерева поиска

3.2.2 Constructor & Destructor Documentation

3.2.2.1 BSTNode()

```
BSTNode::BSTNode (
    const Sportsman & val,
    int idx) [inline]
```

Конструктор, который инициализирует узел на основе спортсмена и его индекса

Parameters

↔ [val]↔	Спортсмен для хранения в узле
↔ [idx]↔	Индекс спортсмена в исходном массиве

3.2.3 Member Data Documentation

3.2.3.1 left

```
BSTNode* BSTNode::left
```

Указатель на левое поддерево

3.2.3.2 originalIndex

```
int BSTNode::originalIndex
```

Индекс в исходном массиве

3.2.3.3 right

```
BSTNode* BSTNode::right
```

Указатель на правое поддерево

3.2.3.4 value

```
Sportsman BSTNode::value
```

Значение узла

The documentation for this struct was generated from the following file:

- [src/search.cpp](#)

3.3 FullName Struct Reference

ФИО спортсмена

```
#include <sportsman.h>
```

Public Member Functions

- bool `operator==` (const [FullName](#) &other) const
Проверяет равенство двух ФИО
- bool `operator<` (const [FullName](#) &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- bool `operator!=` (const [FullName](#) &other) const
Проверяет неравенство двух ФИО
- bool `operator>` (const [FullName](#) &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- bool `operator<=` (const [FullName](#) &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).
- bool `operator>=` (const [FullName](#) &other) const
Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Public Attributes

- `std::string last_name`
- `std::string first_name`
- `std::string middle_name`

3.3.1 Detailed Description

ФИО спортсмена

3.3.2 Member Function Documentation

3.3.2.1 `operator!=()`

```
bool FullName::operator!= (
    const FullName & other) const [inline]
```

Проверяет неравенство двух ФИО

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

`true`, если ФИО не совпадают, иначе `false`

3.3.2.2 `operator<()`

```
bool FullName::operator< (
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

`true`, если текущее ФИО должно идти раньше другого, иначе `false`

3.3.2.3 operator<=()

```
bool FullName::operator<= (
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти не позже другого, иначе false

3.3.2.4 operator==()

```
bool FullName::operator==(
    const FullName & other) const [inline]
```

Проверяет равенство двух ФИО

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если ФИО совпадают, иначе false

3.3.2.5 operator>()

```
bool FullName::operator> (
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти позже другого, иначе false

3.3.2.6 operator>=()

```
bool FullName::operator>= (
    const FullName & other) const [inline]
```

Сравнивает два ФИО для сортировки (сначала по фамилии, затем по имени, затем по отчеству).

Parameters

↔	ФИО для сравнения
[other]↔	

Returns

true, если текущее ФИО должно идти не раньше другого, иначе false

The documentation for this struct was generated from the following file:

- [src/sportsman.h](#)

3.4 HashTable Struct Reference

Хеш-таблица для хранения спортсменов и их индексов

Public Member Functions

- HashTable ()
Конструктор по умолчанию
- void [insert](#) (const [Sportsman](#) &value, int index)
Вставляет спортсмена в хеш-таблицу
- std::vector< int > search (const [Sportsman](#) &target)
- double get_collisions_rate () const

Public Attributes

- `std::vector< std::pair< Sportsman, int > > buckets\_indices [USHRT_MAX]`
- `unsigned int collisions\_cnt = 0`
- `unsigned int total\_insertions = 0`

3.4.1 Detailed Description

Хеш-таблица для хранения спортсменов и их индексов

3.4.2 Member Function Documentation

3.4.2.1 `insert()`

```
void HashTable::insert (
    const Sportsman & value,
    int index) [inline]
```

Вставляет спортсмена в хеш-таблицу

Parameters

<code>↔ [value]↔</code>	Спортсмен для вставки
<code>↔ [index]↔</code>	Индекс спортсмена в исходном массиве

3.4.3 Member Data Documentation

3.4.3.1 `buckets_indices`

```
std::vector<std::pair<Sportsman, int> > HashTable::buckets_indices[USHRT_MAX]
```

Массив бакетов для хранения спортсменов и индексов

3.4.3.2 `collisions_cnt`

```
unsigned int HashTable::collisions_cnt = 0
```

Счетчик коллизий при вставке спортсменов в хеш-таблицу

3.4.3.3 total_insertions

```
unsigned int HashTable::total_insertions = 0
```

Счетчик общего количества вставок спортсменов в хеш-таблицу

The documentation for this struct was generated from the following file:

- [src/search.cpp](#)

3.5 RBNODE Struct Reference

Узел красно-черного дерева

Public Member Functions

- [RBNODE](#) (const [Sportsman](#) &val, int idx)
Конструктор, который инициализирует узел на основе спортсмена

Public Attributes

- [Sportsman](#) value
- int [originalIndex](#)
- [Color](#) color
- [RBNODE](#) * left
- [RBNODE](#) * right
- [RBNODE](#) * parent

3.5.1 Detailed Description

Узел красно-черного дерева

3.5.2 Constructor & Destructor Documentation

3.5.2.1 RNode()

```
RNode::RNode (
    const Sportsman & val,
    int idx) [inline]
```

Конструктор, который инициализирует узел на основе спортсмена

Parameters

 ↔	Спортсмен для хранения в узле
[val]  ↔	

3.5.3 Member Data Documentation

3.5.3.1 color

```
Color RNode::color
```

Цвет узла

3.5.3.2 left

```
RNode* RNode::left
```

Указатель на левое поддерево

3.5.3.3 originalIndex

```
int RNode::originalIndex
```

Индекс спортсмена в исходном массиве

3.5.3.4 parent

```
RNode* RNode::parent
```

Указатель на родителя

3.5.3.5 right

`RBNode* RBNode::right`

Указатель на правое поддерево

3.5.3.6 value

`Sportsman RBNode::value`

Значение узла

The documentation for this struct was generated from the following file:

- `src/search.cpp`

3.6 RedBlackTree Class Reference

Красно-черное дерево

Public Member Functions

- `RedBlackTree ()`
Конструктор, который инициализирует пустое дерево
- `void insert (const Sportsman &value, int index)`
Вставляет спортсмена в красно-черное дерево
- `void rotate_left (RBNode *x)`
Поворачивает узел влево
- `void rotate_right (RBNode *y)`
Поворачивает узел вправо
- `void fix_insert (RBNode *z)`
Исправляет дерево после вставки узла
- `void search_all (RBNode *node, const Sportsman &target, std::vector< int > &result) const`
Ищет спортсмена в поддереве, начиная с данного узла

Public Attributes

- `RBNode * root`

3.6.1 Detailed Description

Красно-черное дерево

3.6.2 Member Function Documentation

3.6.2.1 fix_insert()

```
void RedBlackTree::fix_insert (  
    RBNode * z) [inline]
```

Исправляет дерево после вставки узла

Parameters

↔ [z]↔	Вставленный узел, который может нарушать свойства красно-черного дерева
-----------------------	---

3.6.2.2 insert()

```
void RedBlackTree::insert (  
    const Sportsman & value,  
    int index) [inline]
```

Вставляет спортсмена в красно-черное дерево

Parameters

↔ [value]↔	Спортсмен для вставки
---------------------------	-----------------------

3.6.2.3 rotate_left()

```
void RedBlackTree::rotate_left (  
    RBNode * x) [inline]
```

Поворачивает узел влево

Parameters

↔ [x]↔	Узел для поворота
-----------------------	-------------------

3.6.2.4 rotate_right()

```
void RedBlackTree::rotate_right (
    RBNODE * y) [inline]
```

Поворачивает узел вправо

Parameters

↔	Узел для поворота
[y]↔	

3.6.2.5 search_all()

```
void RedBlackTree::search_all (
    RBNODE * node,
    const Sportsman & target,
    std::vector< int > & result) const [inline]
```

Ищет спортсмена в поддереве, начиная с данного узла

Parameters

↔	Спортсмен для поиска
[target]↔	
↔	Вектор индексов найденных спортсменов, равных target
[result]↔	

3.6.3 Member Data Documentation

3.6.3.1 root

```
RBNODE* RedBlackTree::root
```

Корень дерева

The documentation for this class was generated from the following file:

- [src/search.cpp](#)

3.7 Sportsman Struct Reference

Структура данных, описывающая спортсмена

```
#include <sportsman.h>
```

Public Member Functions

- [Sportsman](#) (std::string csv_string)
Конструктор, который инициализирует поля структуры на основе строки в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).
- std::string [to_csv](#) () const
Вывод в строку в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).
- bool [operator==](#) (const [Sportsman](#) &other) const
Проверяет равенство двух спортсменов (сравнение по ФИО).
- bool [operator<](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по ФИО).
- bool [operator!=](#) (const [Sportsman](#) &other) const
Проверяет неравенство двух спортсменов (сравнение по ФИО).
- bool [operator>](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по ФИО).
- bool [operator<=](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по ФИО).
- bool [operator>=](#) (const [Sportsman](#) &other) const
Сравнивает двух спортсменов для сортировки (сравнение по ФИО).

Public Attributes

- [FullName](#) full_name
- unsigned short [age](#)
- unsigned short [height_cm](#)
- unsigned short [weight_kg](#)
- std::string [sport](#)

3.7.1 Detailed Description

Структура данных, описывающая спортсмена

3.7.2 Constructor & Destructor Documentation

3.7.2.1 Sportsman()

```
Sportsman::Sportsman (
    std::string csv_string) [inline]
```

Конструктор, который инициализирует поля структуры на основе строки в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).

Parameters

<code>[csv_string]</code>	Строка в формате CSV
---------------------------	----------------------

Returns

Инициализированный объект структуры [Sportsman](#)

Exceptions

<code>std::invalid_argument</code> ,если	строка не является корректной CSV строкой или содержит некорректные данные
--	--

3.7.3 Member Function Documentation

3.7.3.1 `operator!=()`

```
bool Sportsman::operator!= (
    const Sportsman & other) const [inline]
```

Проверяет неравенство двух спортсменов (сравнение по ФИО).

Parameters

$\leftrightarrow$ [other] $\leftrightarrow$	Спортсмен для сравнения
--	-------------------------

Returns

true, если спортсмены не совпадают, иначе false

3.7.3.2 `operator<()`

```
bool Sportsman::operator< (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по ФИО).

Parameters

$\leftrightarrow$ [other] $\leftrightarrow$	Спортсмен для сравнения
--	-------------------------

Returns

true, если текущий спортсмен должен идти раньше другого, иначе false

3.7.3.3 operator<=()

```
bool Sportsman::operator<= (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по ФИО).

Parameters

↔ [other]↔	Спортсмен для сравнения
---------------	-------------------------

Returns

true, если текущий спортсмен должен идти не позже другого, иначе false

3.7.3.4 operator==()

```
bool Sportsman::operator== (
    const Sportsman & other) const [inline]
```

Проверяет равенство двух спортсменов (сравнение по ФИО).

Parameters

↔ [other]↔	Спортсмен для сравнения
---------------	-------------------------

Returns

true, если спортсмены совпадают, иначе false

3.7.3.5 operator>()

```
bool Sportsman::operator> (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по ФИО).

Parameters

↔	Спортсмен для сравнения
[other]↔	

Returns

true, если текущий спортсмен должен идти позже другого, иначе false

3.7.3.6 operator>=()

```
bool Sportsman::operator>= (
    const Sportsman & other) const [inline]
```

Сравнивает двух спортсменов для сортировки (сравнение по ФИО).

Parameters

↔	Спортсмен для сравнения
[other]↔	

Returns

true, если текущий спортсмен должен идти не раньше другого, иначе false

3.7.3.7 to_csv()

```
std::string Sportsman::to_csv () const [inline]
```

Вывод в строку в формате CSV (вид спорта, фамилия, имя, отчество, возраст, рост, вес).

Returns

Строка в формате CSV

3.7.4 Member Data Documentation

3.7.4.1 age

```
unsigned short Sportsman::age
```

Возраст спортсмена в годах

3.7.4.2 full_name

`FullName Sportsman::full_name`

ФИО спортсмена

3.7.4.3 height_cm

`unsigned short Sportsman::height_cm`

Рост спортсмена в сантиметрах

3.7.4.4 sport

`std::string Sportsman::sport`

Спорт, которым занимается спортсмен

3.7.4.5 weight_kg

`unsigned short Sportsman::weight_kg`

Вес спортсмена в килограммах

The documentation for this struct was generated from the following file:

- [src/sportsman.h](#)

Chapter 4

File Documentation

4.1 src/search.cpp File Reference

Реализация методов поиска спортсменов в массиве

```
#include "search.h"
#include "sportsman.h"
#include <climits>
#include <map>
#include <vector>
```

Classes

- struct [BSTNode](#)
Узел двоичного дерева поиска
- struct [BinarySearchTree](#)
Двоичное дерево поиска
- struct [RBNode](#)
Узел красно-черного дерева
- class [RedBlackTree](#)
Красно-черное дерево
- struct [HashTable](#)
Хеш-таблица для хранения спортсменов и их индексов

Enumerations

- enum [Color](#) { [RED](#) , [BLACK](#) }

Functions

- `std::vector< int > linear_search (const std::vector< Sportsman > &sportsmen, const Sportsman &target)`
- `std::vector< int > binary_tree_search (const std::vector< Sportsman > &sportsmen, const Sportsman &target)`
- `std::vector< int > red_black_tree_search (const std::vector< Sportsman > &sportsmen, const Sportsman &target)`
- `unsigned short hash\_sportsman (const Sportsman &value)`
Функция хэширования спортсмена
- `std::vector< int > hash_table_search (const std::vector< Sportsman > &sportsmen, const Sportsman &target, unsigned int &collisions, unsigned int &total_insertions)`
- `std::vector< int > multimap_search (const std::vector< Sportsman > &sportsmen, const Sportsman &target)`

4.1.1 Detailed Description

Реализация методов поиска спортсменов в массиве

4.1.2 Enumeration Type Documentation

4.1.2.1 Color

enum [Color](#)

Enumerator

RED	Красный цвет узла
BLACK	Черный цвет узла

4.1.3 Function Documentation

4.1.3.1 [hash_sportsman\(\)](#)

```
unsigned short hash_sportsman (
    const Sportsman & value)
```

Функция хэширования спортсмена

Parameters

↔ [value]↔	Спортсмен для хэширования
---------------	---------------------------

Returns

Хэш-значение ФИО спортсмена

4.2 src/search.h File Reference

Объявление методов поиска спортсменов в массиве

```
#include "sportsman.h"
#include <vector>
```

Functions

- `std::vector< int > linear_search (const std::vector< Sportsman > &sportsmen, const Sportsman &target)`
- `std::vector< int > binary_tree_search (const std::vector< Sportsman > &sportsmen, const Sportsman &target)`
- `std::vector< int > red_black_tree_search (const std::vector< Sportsman > &sportsmen, const Sportsman &target)`
- `std::vector< int > hash_table_search (const std::vector< Sportsman > &sportsmen, const Sportsman &target, unsigned int &collisions, unsigned int &total_insertions)`
- `std::vector< int > multimap_search (const std::vector< Sportsman > &sportsmen, const Sportsman &target)`

4.2.1 Detailed Description

Объявление методов поиска спортсменов в массиве

4.3 search.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "sportsman.h"
00009 #include <vector>
00010
00011 std::vector<int> linear_search(const std::vector<Sportsman> &sportsmen, const Sportsman &target);
00012
00013 std::vector<int> binary_tree_search(const std::vector<Sportsman> &sportsmen, const Sportsman &target);
00014
00015 std::vector<int> red_black_tree_search(const std::vector<Sportsman> &sportsmen, const Sportsman
&target);
00016
00017 std::vector<int> hash_table_search(const std::vector<Sportsman> &sportsmen, const Sportsman &target,
unsigned int &collisions, unsigned int &total_insertions);
00018
00019 std::vector<int> multimap_search(const std::vector<Sportsman> &sportsmen, const Sportsman &target);
```

4.4 src/sportsman.h File Reference

Структура данных, описывающая спортсмена

```
#include <stdexcept>
#include <string>
```

Classes

- struct [FullName](#)
ФИО спортсмена
- struct [Sportsman](#)
Структура данных, описывающая спортсмена

4.4.1 Detailed Description

Структура данных, описывающая спортсмена

4.5 sportsman.h

[Go to the documentation of this file.](#)

```

00001
00005
00006 #pragma once
00007
00008 #include <stdexcept>
00009 #include <string>
00010
00015 struct FullName
00016 {
00017     std::string last_name;    /**< Фамилия */
00018     std::string first_name;   /**< Имя */
00019     std::string middle_name;  /**< Отчество */
00020
00026     bool operator==(const FullName &other) const
00027     {
00028         return first_name == other.first_name &&
00029             middle_name == other.middle_name &&
00030             last_name == other.last_name;
00031     }
00032
00038     bool operator<(const FullName &other) const
00039     {
00040         if (last_name != other.last_name)
00041         {
00042             return last_name < other.last_name;
00043         }
00044         if (first_name != other.first_name)
00045         {
00046             return first_name < other.first_name;
00047         }
00048         return middle_name < other.middle_name;
00049     }
00050
00056     bool operator!=(const FullName &other) const
00057     {
00058         return !(*this == other);
00059     }
00060
00066     bool operator>(const FullName &other) const
00067     {
00068         return other < *this;
00069     }
00070
00076     bool operator<=(const FullName &other) const
00077     {
00078         return !(other < *this);
00079     }
00080
00086     bool operator>=(const FullName &other) const
00087     {
00088         return !(*this < other);
00089     }
00090 };
00091
00096 struct Sportsman
00097 {

```

```

00098     FullName full_name;
00099     unsigned short age;
00100     unsigned short height_cm;
00101     unsigned short weight_kg;
00102     std::string sport;
00103
00110     Sportsman(std::string csv_string)
00111     {
00112         int pos = 0;
00113         std::string token;
00114         std::string delimiter = ",";
00115         int field_index = 0;
00116
00117         for (int field_idx = 0; field_idx < 7; field_idx++)
00118         {
00119             pos = csv_string.find(delimiter);
00120             if (pos == std::string::npos)
00121             {
00122                 if (field_idx != 6)
00123                 {
00124                     throw std::invalid_argument("Недостаточно полей в CSV строке");
00125                 }
00126                 token = csv_string;
00127             }
00128             else if (field_idx == 6)
00129             {
00130                 throw std::invalid_argument("Слишком много полей в CSV строке");
00131             }
00132             else
00133             {
00134                 token = csv_string.substr(0, pos);
00135                 csv_string.erase(0, pos + delimiter.length());
00136             }
00137
00138             switch (field_idx)
00139             {
00140                 case 0:
00141                     sport = token;
00142                     break;
00143                 case 1:
00144                     full_name.last_name = token;
00145                     break;
00146                 case 2:
00147                     full_name.first_name = token;
00148                     break;
00149                 case 3:
00150                     full_name.middle_name = token;
00151                     break;
00152                 case 4:
00153                     age = (unsigned short)std::stoul(token);
00154                     break;
00155                 case 5:
00156                     height_cm = (unsigned short)std::stoul(token);
00157                     break;
00158                 case 6:
00159                     weight_kg = (unsigned short)std::stoul(token);
00160                     break;
00161             }
00162         }
00163     }
00164
00169     std::string to_csv() const
00170     {
00171         return sport + "," +
00172             full_name.last_name + "," +
00173             full_name.first_name + "," +
00174             full_name.middle_name + "," +
00175             std::to_string(age) + "," +
00176             std::to_string(height_cm) + "," +
00177             std::to_string(weight_kg);
00178     }
00179
00185     bool operator==(const Sportsman &other) const
00186     {
00187         return full_name == other.full_name;
00188     }
00189
00195     bool operator<(const Sportsman &other) const
00196     {
00197         return full_name < other.full_name;
00198     }
00199
00205     bool operator!=(const Sportsman &other) const
00206     {
00207         return !(*this == other);
00208     }
00209

```

```

00215     bool operator>(const Sportsman &other) const
00216     {
00217         return other < *this;
00218     }
00219
00225     bool operator<=(const Sportsman &other) const
00226     {
00227         return !(other < *this);
00228     }
00229
00235     bool operator>=(const Sportsman &other) const
00236     {
00237         return !(*this < other);
00238     }
00239 };

```

4.6 src/utils.h File Reference

Вспомогательные функции

```

#include "sportsman.h"
#include <chrono>
#include <functional>
#include <iostream>
#include <string>
#include <vector>

```

Functions

- `std::vector< Sportsman > from_csv_file (const std::string &filename)`
Прочитать вектор спортсменов из CSV файла
- `void to_csv_file (const std::vector< Sportsman > &data, const std::string &filename)`
Записать вектор спортсменов в CSV-файл
- `double measure_time_ms (std::function< std::vector< int >(std::vector< Sportsman > &, const Sportsman &)> find_func, const std::vector< Sportsman > &source, const Sportsman &target)`
Измерение времени поиска

4.6.1 Detailed Description

Вспомогательные функции

4.6.2 Function Documentation

4.6.2.1 from_csv_file()

```

std::vector< Sportsman > from_csv_file (
    const std::string & filename)

```

Прочитать вектор спортсменов из CSV файла

Parameters

filename	Путь к входному файлу
----------	-----------------------

Returns

Вектор [Sportsman](#)

4.6.2.2 measure_time_ms()

```
double measure_time_ms (  
    std::function< std::vector< int >(std::vector< Sportsman > &, const Sportsman &)>  
    find_func,  
    const std::vector< Sportsman > & source,  
    const Sportsman & target)
```

Измерение времени поиска

Parameters

find_func	Функция поиска (std::vector<int>(std::vector<Sportsman>&))
source	Вектор спортсменов для поиска

Returns

Время поиска (в миллисекундах)

4.6.2.3 to_csv_file()

```
void to_csv_file (  
    const std::vector< Sportsman > & data,  
    const std::string & filename)
```

Записать вектор спортсменов в CSV-файл

Parameters

data	Вектор Sportsman
filename	Путь к выходному файлу

4.7 utils.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #pragma once
00007
00008 #include "sportsman.h"
00009 #include <chrono>
00010 #include <functional>
00011 #include <iostream>
00012 #include <string>
00013 #include <vector>
00014
00020 std::vector<Sportsman> from_csv_file(const std::string &filename);
00021
00027 void to_csv_file(const std::vector<Sportsman> &data, const std::string &filename);
00028
00035 double measure_time_ms(std::function<std::vector<int>(std::vector<Sportsman> &, const Sportsman &)>
    find_func, const std::vector<Sportsman> &source, const Sportsman &target);
```


Index

- age
 - Sportsman, [22](#)
- BinarySearchTree, [5](#)
 - insert, [6](#)
 - insertRec, [6](#)
 - root, [7](#)
 - search, [6](#)
 - searchRec, [7](#)
- BLACK
 - search.cpp, [26](#)
- BSTNode, [8](#)
 - BSTNode, [8](#)
 - left, [8](#)
 - originalIndex, [8](#)
 - right, [9](#)
 - value, [9](#)
- buckets_indices
 - HashTable, [13](#)
- collisions_cnt
 - HashTable, [13](#)
- Color
- search.cpp, [26](#)
- color
 - RBNode, [15](#)
- fix_insert
 - RedBlackTree, [17](#)
- from_csv_file
 - utils.h, [30](#)
- full_name
 - Sportsman, [22](#)
- FullName, [9](#)
 - operator!=, [10](#)
 - operator<, [10](#)
 - operator<=, [10](#)
 - operator>, [11](#)
 - operator>=, [12](#)
 - operator==, [11](#)
- hash_sportsman
 - search.cpp, [26](#)
- HashTable, [12](#)
 - buckets_indices, [13](#)
 - collisions_cnt, [13](#)
 - insert, [13](#)
 - total_insertions, [13](#)
- height_cm
 - Sportsman, [23](#)
- insert
 - BinarySearchTree, [6](#)
 - HashTable, [13](#)
 - RedBlackTree, [17](#)
- insertRec
 - BinarySearchTree, [6](#)
- left
 - BSTNode, [8](#)
 - RBNode, [15](#)
- measure_time_ms
 - utils.h, [31](#)
- operator!=
 - FullName, [10](#)
 - Sportsman, [20](#)
- operator<
 - FullName, [10](#)
 - Sportsman, [20](#)
- operator<=
 - FullName, [10](#)
 - Sportsman, [20](#)
- operator>
 - FullName, [11](#)
 - Sportsman, [21](#)
- operator>=
 - FullName, [12](#)
 - Sportsman, [22](#)
- operator==
 - FullName, [11](#)
 - Sportsman, [21](#)
- originalIndex
 - BSTNode, [8](#)
 - RBNode, [15](#)
- parent
 - RBNode, [15](#)
- RBNode, [14](#)
 - color, [15](#)
 - left, [15](#)
 - originalIndex, [15](#)
 - parent, [15](#)
 - RBNode, [15](#)
 - right, [15](#)
 - value, [16](#)
- RED
 - search.cpp, [26](#)
- RedBlackTree, [16](#)

- fix_insert, [17](#)
- insert, [17](#)
- root, [18](#)
- rotate_left, [17](#)
- rotate_right, [17](#)
- search_all, [18](#)
- right
 - BSTNode, [9](#)
 - RBNode, [15](#)
- root
 - BinarySearchTree, [7](#)
 - RedBlackTree, [18](#)
- rotate_left
 - RedBlackTree, [17](#)
- rotate_right
 - RedBlackTree, [17](#)
- search
 - BinarySearchTree, [6](#)
- search.cpp
 - BLACK, [26](#)
 - Color, [26](#)
 - hash_sportsman, [26](#)
 - RED, [26](#)
- search_all
 - RedBlackTree, [18](#)
- searchRec
 - BinarySearchTree, [7](#)
- sport
 - Sportsman, [23](#)
- Sportsman, [18](#)
 - age, [22](#)
 - full_name, [22](#)
 - height_cm, [23](#)
 - operator!=, [20](#)
 - operator<, [20](#)
 - operator<=, [20](#)
 - operator>, [21](#)
 - operator>=, [22](#)
 - operator==, [21](#)
 - sport, [23](#)
 - Sportsman, [19](#)
 - to_csv, [22](#)
 - weight_kg, [23](#)
- src/search.cpp, [25](#)
- src/search.h, [27](#)
- src/sportsman.h, [27](#), [28](#)
- src/utils.h, [30](#), [32](#)
- to_csv
 - Sportsman, [22](#)
- to_csv_file
 - utils.h, [31](#)
- total_insertions
 - HashTable, [13](#)
- utils.h
 - from_csv_file, [30](#)
 - measure_time_ms, [31](#)
 - to_csv_file, [31](#)
 - value
 - BSTNode, [9](#)
 - RBNode, [16](#)
 - weight_kg
 - Sportsman, [23](#)